

CSE 3207 — Theory Notes

11 Important Questions — Class Test 1 Revision Sheet

1. Agent Concept — Explain with Examples

Definition of Agent

An **agent** is anything that interacts with its environment through **sensors and actuators**.

Basic structure:

Environment → **Sensors** → **Agent** → **Actuators** → **Environment**

- **Sensors:** receive percepts from the environment.
- **Actuators:** perform actions on the environment.
- **Percepts:** information received by the agent from the environment.
- **Actions:** things performed by the agent.

Examples of Agents

Humans, Robots, Softbots, Thermostats.

Example: Vacuum Cleaner World

Percepts: [Location, Status] — e.g. [1, Clean], [2, Dirty], ...

Actions: PickDirt, MoveToRight, Halt

Sensors: Dirt finder, Location detector

Actuators: Suction pump, Locomotive

Agent Program

Agent1([Location, Status])

The program works as:

- If Status = Dirty → **PickDirt**
- Else if Location = End → **Halt**
- Else → **MoveToRight**

So, an agent receives **percepts**, processes them through its agent program, and produces **actions**.

2. Characteristics of a Rational Agent

A **rational agent** chooses actions that **optimize expected performance**, given its percept sequence and knowledge base at the time.

Characteristics

- 1 **Optimizes expected performance** — chooses actions expected to give the best performance.
- 2 **Learns** — improves performance based on experience.
- 3 **Functions autonomously** — operates without constant external control.

Basic Types of Agent Programs

Type	Example
Simple reflex agent	VacuumCleaner001
Model-based reflex agent	GateMan77
Goal-based agent	Planner999
Utility-based agent	ChessMaster111

Exam Answer

A rational agent chooses actions that optimize expected performance given the percept sequence and knowledge base at the time. It learns and improves its performance based on experience and functions autonomously.

3. Environment Concerns for Rational Agent Design

Six major environment contrasts to consider:

1. Fully Observable / Partially Observable

2. Deterministic / Stochastic

3. Episodic / Sequential

4. Static / Dynamic

5. Discrete / Continuous

6. Single Agent / Multiple Agent

For your exam: if the question says "describe," write the six pairs and give a one-line meaning for each if you have enough marks/time.

4. Forward & Backward Chaining (Inference in PL)

PL = Propositional Logic, used for representing Knowledge Bases (KBs), reasoning/inference, and answering queries. **Inference** means deriving a new sentence from old ones. The KB is assumed to contain **definite Horn clauses** representing rules and facts.

A. Forward Chaining — Data-Driven

Query → Data Search → Premise → Conclusion → Conclusion added to KB → ...

It continues until the query is found or the KB is exhausted.

Example

Facts: A, B, C, D. Rules: $A \wedge B \Rightarrow L$, $A \wedge D \Rightarrow G$, $B \wedge L \Rightarrow M$. Query: KB ■ M ?

Step 1: A, B and $A \wedge B \Rightarrow L \Rightarrow L$ is added to the KB.

Step 2: B, L and $B \wedge L \Rightarrow M \Rightarrow M$ is added to the KB.

M is proved.

B. Backward Chaining — Goal-Oriented

Query → Goal → Conclusion → Premise → Subgoal → ... (with backtracking)

Example

Query: KB ■ M ? Start with ?M. Find rule $B \wedge L \Rightarrow M$, so need ? $B \wedge L$. B is known; need L. Find $A \wedge P \Rightarrow L \Rightarrow$ need ? $A \wedge P$. A is known but P needs another rule — backtrack. Eventually find $A \wedge B \Rightarrow L$; since A and B are known, L holds. Then $B \wedge L \Rightarrow M$ gives M.

M is proved.

Main Difference

Forward Chaining	Backward Chaining
Data-driven	Goal-oriented
Starts from known facts	Starts from query
Moves toward conclusions	Moves toward premises/subgoals

5. Limitations of Logical Inference

Example: “Treasure Hunt: A Risky Business — Find Gold avoiding Monsters”

Problem

4 × 4 grid

There may be a monster in any cell except [1,1]. The probability of a monster in a cell is greater than 0 and independent. Gold is hidden in some cells. The agent must find gold while avoiding monsters.

Predicates

- $A_{i,j}$ = Agent is in cell $[i,j]$
- $\neg M_{i,j}$ = No monster is in cell $[i,j]$
- $G_{i,j}$ = Gold glitters in cell $[i,j]$
- $S_{i,j}$ = Smell of a monster is perceived in cell $[i,j]$

Important Rule

$$S_{i,j} \leftrightarrow M_{i-1,j} \vee M_{i+1,j} \vee M_{i,j-1} \vee M_{i,j+1}$$

A monster smell indicates a monster may be in a neighboring cell.

Where Is the Limitation?

Suppose the agent perceives $S_{2,1}$. Reasoning establishes $M_{2,2} \vee M_{3,1}$, but this does **not** tell us exactly which cell contains the monster.

Later, $M_{1,2} \vee M_{2,2} \vee M_{2,1}$ may hold, but the combination is uncertain.

Limitation: logical inference may not provide enough information to determine one definite state when several possibilities remain.

6. Syntactic Rules of First Order Logic (FOL)

1. Sentence

Sentence $\rightarrow$ AtomicSentence | $\neg$ Sentence | (Sentence) | Sentence Connective Sentence | Quantifier Variable,...Variable Sentence

2. Atomic Sentence

AtomicSentence $\rightarrow$ Predicate(Term,Term,...) | Term = Term

3. Term

Term $\rightarrow$ Constant | Variable | Function(Term,...)

A term represents an **object**. A complex term represents a complicated object. A ground term is a term **without** a **variable**.

4. Connectives

$\wedge, \vee, \Rightarrow, \Leftrightarrow$

5. Quantifiers

$\forall, \exists$

6. Variables

Examples: x, y, z, ...

7. Predicates

Examples: Father, Subset, Greater, ...

8. Constants

Examples: Karim, AUST, X1, ...

9. Functions

Examples: Sum, Log, MotherOf, ...

Examples of Syntactically Correct FOL Sentences

MotherOf(Rina) = Bina

$\forall x,y,z ((\text{Father}(x,y) \wedge \text{Father}(y,z)) \Rightarrow \text{Grandfather}(x,z))$

7. Semantics of a Query to a KB in FOL

Semantics is used to **interpret expressions (sentences)**, i.e. to find truth values or matches by fixing substitutions or binding lists. A binding list consists of:

variable / term pairs

Example

Knowledge Base:

Father(Karim, Rahim) Father(Rahim, Selim) Father(Rahim, Halim)

$\forall x,y,z ((\text{Father}(x,y) \wedge \text{Father}(y,z)) \Rightarrow \text{Grandfather}(x,z))$

Query 1

ASK(KB, Father(Rahim, Selim)) — “Is Rahim the father of Selim?”

Result: True

Query 2

ASK(KB, $\exists x,y \text{ Grandfather}(x,y)$) — “Who are grandfather of whom?”

The KB returns substitutions: $x/\text{Karim}, y/\text{Selim}$ and $x/\text{Karim}, y/\text{Halim}$

So a query can return **True/False** or **substitutions/bindings**, depending on the query.

8. Unification & the UNIFY Algorithm

Definition

Unification is the process of making sentences equal using **substitution**. A **unifier** is a substitution that makes sentences equal. A **Most General Unifier (MGU)** is a unifier that is not an instance of any other unifier.

Basic Matching Rules

- 1 Different predicates do not match.
- 2 Different constants do not match.
- 3 A variable matches a term if the term does not contain the variable itself.

Example

$S = \{ P1("Km", x, y), P1(z, u, F1(z)) \}$

Step 1

Initially $\theta = \{\}$, and $SUBST(\theta, S) = S$

Step 2

Match $z = "Km" \Rightarrow \theta = \{ z/"Km" \}$

$SUBST(\theta, S)$ becomes $\{ P1("Km", x, y), P1("Km", u, F1("Km")) \}$

Step 3

Match $x = u \Rightarrow \theta = \{ z/"Km", x/u \}$

$\Rightarrow \{ P1("Km", u, y), P1("Km", u, F1("Km")) \}$

Step 4

Match $y = F1("Km") \Rightarrow \theta = \{ z/"Km", x/u, y/F1("Km") \}$

Now $SUBST(\theta, S) = \{ P1("Km", u, F1("Km")) \}$ — both sentences equal, algorithm terminates.

MGU = $\{ z/"Km", x/u, y/F1("Km") \}$

9. Natural Language $\rightarrow$ CNF of FOL

Example: "Every guest receives at least one gift."

Step 1 — Natural Language $\rightarrow$ FOL

$\forall x (Guest(x) \Rightarrow \exists y (Gift(y) \wedge Receives(x,y)))$

Step 2 — Standardize Variables

Use separate variables for different quantifiers.

Step 3 — Eliminate Implication, Move Negation Inward

Using $A \Rightarrow B \equiv \neg A \vee B$:

$\forall x (\neg \text{Guest}(x) \vee \exists y (\text{Gift}(y) \wedge \text{Receives}(x,y)))$

Step 4 — Skolemize

Replace the existential variable y with the Skolem function $\text{GiftFor}(x)$:

$\forall x (\neg \text{Guest}(x) \vee (\text{Gift}(\text{GiftFor}(x)) \wedge \text{Receives}(x, \text{GiftFor}(x))))$

Step 5 — Drop All Universal Quantifiers

$\neg \text{Guest}(x) \vee (\text{Gift}(\text{GiftFor}(x)) \wedge \text{Receives}(x, \text{GiftFor}(x)))$

Step 6 — Simplify to CNF (Distribute OR over AND)

$(\neg \text{Guest}(x) \vee \text{Gift}(\text{GiftFor}(x))) \wedge (\neg \text{Guest}(x) \vee \text{Receives}(x, \text{GiftFor}(x)))$

Two CNF clauses:

1. $\neg \text{Guest}(x) \vee \text{Gift}(\text{GiftFor}(x))$

2. $\neg \text{Guest}(x) \vee \text{Receives}(x, \text{GiftFor}(x))$

CNF Conversion Sequence to Memorize

Standardize $\rightarrow$ Eliminate $\Rightarrow, \Leftrightarrow \rightarrow$ Move $\neg$ inward $\rightarrow$ Skolemize $\rightarrow$ Drop $\forall \rightarrow$ Simplify to CNF

10. Resolution Principle (Proving a Query)

Resolution Principle

Resolution works on **CNF clauses**. Given two clauses containing complementary literals, we:

- 1 Unify the complementary literals.
- 2 Remove those complementary literals.
- 3 Combine the remaining literals.
- 4 Apply the resulting substitution.

Represented as $\theta = \text{UNIFY}(p_k, \neg q_l)$, then apply $\text{SUBST}(\theta, \dots)$ to the remaining literals.

Example

KB in CNF:

1. $\neg \text{Guest}(x) \vee \text{Gift}(\text{GiftFor}(x))$
2. $\neg \text{Guest}(x) \vee \text{Receives}(x, \text{GiftFor}(x))$
3. $\text{Guest}(\text{Karim})$

Query: Does Karim receive a gift? $\exists x (\text{Gift}(x) \wedge \text{Receives}(\text{Karim}, x))$

Step 1 — Negate the Query

$\neg \exists x (\text{Gift}(x) \wedge \text{Receives}(\text{Karim}, x)) \rightarrow \forall x \neg (\text{Gift}(x) \wedge \text{Receives}(\text{Karim}, x))$

By De Morgan: $\forall x (\neg \text{Gift}(x) \vee \neg \text{Receives}(\text{Karim}, x))$

Drop $\forall$:

T1: $\neg \text{Gift}(y) \vee \neg \text{Receives}(\text{Karim}, y)$ — added to KB

Step 2 — Resolve T1 with Clause 1

Unify $y = \text{GiftFor}(x)$:

T2: $\neg \text{Guest}(x) \vee \neg \text{Receives}(\text{Karim}, \text{GiftFor}(x))$

Step 3 — Resolve T1 with Clause 2

Unify $x = \text{Karim}, y = \text{GiftFor}(\text{Karim})$:

T3: $\neg \text{Guest}(\text{Karim}) \vee \neg \text{Gift}(\text{GiftFor}(\text{Karim}))$

Step 4 — Resolve T3 with Clause 1

T4: $\neg \text{Guest}(\text{Karim})$

Step 5 — Resolve T4 with Clause 3

$\text{Guest}(\text{Karim})$ and $\neg \text{Guest}(\text{Karim})$ resolve to:

[] — the empty clause

The empty clause has been derived $\Rightarrow$ the negation of the query leads to a contradiction $\Rightarrow$ the query is TRUE. Yes, Karim receives a gift.

11. Resolution-Refutation Completeness

The refutation procedure used above, in general form. Suppose the query is Q :

- 1 **Step 1:** Take the negation, $\neg Q$
- 2 **Step 2:** Convert $\neg Q$ into CNF
- 3 **Step 3:** Add the CNF of $\neg Q$ to the KB
- 4 **Step 4:** Apply the Resolution rule repeatedly
- 5 **Step 5:** Try to derive the empty clause []

If the Empty Clause IS Derived

$KB \wedge \neg Q$ leads to a contradiction, therefore:

Q is TRUE

If the Empty Clause CANNOT Be Derived

Q is FALSE

This process is called the **Resolution-Refutation completeness of KB**.

Exam-Ready Answer

Resolution-refutation completeness is the process of proving a query by adding the CNF of its negation to the KB and repeatedly applying the Resolution rule. If an empty clause [] is derived, a contradiction is obtained and the query is proved true. If the empty clause cannot be derived, the query is false according to the given course material.

■ Super-Short Revision Sheet

Q1 — Agent

Sensors → Agent → Actuators (Example: Vacuum Cleaner)

Q2 — Rational Agent

Optimize expected performance + Learn + Autonomous

Q3 — Environment

FO/PO | Deterministic/Stochastic | Episodic/Sequential | Static/Dynamic | Discrete/Continuous
| Single/Multi-Agent

Q4 — PL Inference

Forward chaining: Data-driven | Backward chaining: Goal-oriented

Q5 — Limitation

Find Gold avoiding Monsters:

$M_{1,3} \vee M_{2,2} \vee M_{3,1}$ may hold, but the combination is uncertain.

Q6 — FOL Syntax

Sentence, AtomicSentence, Term, Connective, Quantifier

Term → Constant | Variable | Function(Term,...)

Q7 — FOL Semantics

Interpret sentences → Truth values / Bindings

Q8 — Unification

Make sentences equal using substitution

$MGU = \{ z/“Km”, x/u, y/F1(“Km”) \}$

Q9 — CNF

Standardize → Eliminate → Move $\neg$ → Skolemize → Drop $\forall$ → CNF

Q10 — Resolution

Negate Query → CNF → Add to KB → Resolve → []

[] ⇒ Query proved

Q11 — Resolution Refutation

KB + $\neg Q \rightarrow$ Resolution $\rightarrow [] \Rightarrow Q$ is TRUE